

150-DAY TRANSITION PLAN

Java Developer → AI Engineer → LLM Engineer

// five months, five modules, one commit at a time



150	5	15	15
DAYS	MONTHS	SPRINTS	SKILL AREAS

A structured, day-by-day path from backend Java engineering into modern AI & LLM engineering.

Modules Covered

Python

Math

Machine Learning

Deep Learning

PyTorch

NLP

Transformers

LLMs

Embeddings

Vector DB

RAG

FastAPI

Agents

Fine-tuning

MLOps

Monthly Roadmap

MONTH	FOCUS	OUTCOME
01	Python, Math, Machine Learning	Ship classical ML pipelines confidently
02	Deep Learning, PyTorch, NLP	Train CNNs/RNNs and understand embeddings
03	Transformers, LLM Internals	Understand attention & how GPT/BERT actually work
04	Embeddings, Vector DB, RAG	Build a production-grade RAG chatbot
05	Agents, FastAPI, Fine-tuning, MLOps	Ship a deployed, agentic AI product

How To Use This Roadmap

1

One sprint at a time
Each month breaks into 3 ten-day sprints. Finish one before starting the next.

2

Tick as you go
Use the checkbox next to each day to mark it complete and track your streak.

3

Ship the mini-projects
Every sprint ends with a project — that's the proof you actually learned it.

01

Month 1 — Python, Math & ML

DAYS 001–030 · PYTHON, MATH, MACHINE LEARNING

→ Ship classical ML pipelines confidently

SPRINT 01 · DAYS 001–010 · PYTHON FUNDAMENTALS

☐	D001	Python setup, syntax & variables	Install Python, set up your editor, run your first scripts.
☐	D002	Control flow — if/else, loops, comprehensions	Rewrite 5 loop-based snippets as comprehensions.
☐	D003	Functions, *args/**kwargs, scope	Write a small utility library of reusable functions.
☐	D004	Data structures deep dive	Lists, tuples, dicts, sets — practice 10 manipulation drills.
☐	D005	OOP in Python	Classes & inheritance, compared directly against Java's model.
☐	D006	Exception handling & context managers	Use try/except/finally and the `with` statement properly.
☐	D007	File I/O, JSON, requests	Read/write files and call a public REST API.
☐	D008	Modules, packages, venvs, pip	Structure a multi-file project with a virtual environment.
☐	D009	Decorators, generators, iterators	Write a custom decorator and a generator-based pipeline.
☐	D010	Mini project + review	Build a CLI expense tracker. Review week 1 concepts.

SPRINT 02 · DAYS 011–020 · NUMPY, PANDAS & LINEAR ALGEBRA

☐	D011	NumPy arrays, indexing, broadcasting	Practice slicing, reshaping and broadcasting rules.
☐	D012	Vectorized ops, random, linspace	Replace 5 Python loops with vectorized NumPy code.
☐	D013	Linear algebra basics	Vectors, matrices, dot product — by hand and in NumPy.
☐	D014	Matrix operations	Transpose, inverse, determinant, and why they matter for ML.
☐	D015	Eigenvalues & eigenvectors	Build intuition with 2D visualizations.
☐	D016	Pandas Series & DataFrame basics	Load a CSV and explore rows, columns, dtypes.
☐	D017	Pandas — filter, group, merge	Practice groupby, merge/join, and pivot tables.
☐	D018	Data cleaning & EDA	Handle missing values, outliers, and duplicate records.
☐	D019	Matplotlib / Seaborn basics	Plot distributions, correlations, and trends.
☐	D020	Mini project + review	Full EDA report on a real dataset (Titanic or housing prices).

SPRINT 03 · DAYS 021–030 · MACHINE LEARNING BASICS

☐	D021	Supervised vs unsupervised vs RL	Map ML problem types to real-world use cases.
☐	D022	Linear Regression from scratch	Implement gradient descent for linear regression in NumPy.
☐	D023	Linear Regression with scikit-learn	Compare your from-scratch model to sklearn's.
☐	D024	Logistic Regression & classification metrics	Precision, recall, F1, ROC-AUC — compute them by hand.
☐	D025	Decision Trees & Random Forests	Train and visualize a decision tree; tune a forest.
☐	D026	SVM & KNN	Implement both on the same dataset and compare boundaries.
☐	D027	Model evaluation	Cross-validation, train/test split, over/underfitting.
☐	D028	Feature engineering & scaling	Standardization, encoding categoricals, feature selection.
☐	D029	Clustering — K-Means	Unsupervised segmentation on a real dataset.
☐	D030	Mini project + review	End-to-end ML pipeline on a Kaggle dataset, month 1 recap.

02

Month 2 — Deep Learning, PyTorch, NLP

DAYS 031–060 · DEEP LEARNING, PYTORCH, NLP

→ Train CNNs/RNNs and understand embeddings

SPRINT 04 · DAYS 031–040 · NEURAL NETWORKS

☐	D031 The perceptron	Biological neuron analogy → single-layer perceptron in NumPy.
☐	D032 Forward propagation math	Trace the math for a 2-layer network by hand.
☐	D033 Activation functions	Implement ReLU, sigmoid, softmax and plot their curves.
☐	D034 Loss functions & gradient descent	MSE, cross-entropy, and gradient descent intuition.
☐	D035 Backpropagation — the chain rule	Derive backprop for a tiny network step by step.
☐	D036 Intro to PyTorch — tensors	Tensor creation, ops, GPU vs CPU device basics.
☐	D037 PyTorch autograd	Understand <code>.backward()</code> and computation graphs.
☐	D038 Your first neural net in PyTorch	Build a feedforward net with <code>nn.Module</code> .
☐	D039 Training loop & optimizers	SGD vs Adam — write a full train/eval loop.
☐	D040 Mini project + review	MNIST digit classifier from scratch in PyTorch.

SPRINT 05 · DAYS 041–050 · CNN, RNN & LSTM

☐	D041 Convolution operation intuition	Visualize filters sliding over an image manually.
☐	D042 CNN architecture	Filters, pooling, stride, padding — build the intuition.
☐	D043 Build a CNN in PyTorch	Image classifier on CIFAR-10 or a custom dataset.
☐	D044 Transfer learning	Fine-tune a pretrained ResNet on your own images.
☐	D045 Sequence data & why RNNs	Understand what breaks feedforward nets on sequences.
☐	D046 RNN architecture & vanishing gradients	Implement a vanilla RNN and see it fail on long sequences.
☐	D047 LSTM & GRU internals	Gates explained — forget, input, output.
☐	D048 Build an LSTM text classifier	Sentiment classification with an LSTM in PyTorch.
☐	D049 Time-series / sequence project	Apply RNN/LSTM to a forecasting or tagging task.
☐	D050 Mini project + review	Ship an image or text classifier. Month 2 recap.

SPRINT 06 · DAYS 051–060 · NLP + EMBEDDINGS

☐	D051 NLP pipeline basics	Tokenization, stemming, lemmatization with spaCy/NLTK.
☐	D052 Bag of Words & TF-IDF	Vectorize text and inspect the resulting feature space.
☐	D053 Word2Vec intuition	Skip-gram vs CBOW — train a tiny Word2Vec model.
☐	D054 GloVe & pretrained embeddings	Load pretrained vectors and explore word analogies.
☐	D055 Cosine similarity & semantic search	Rank sentences by embedding similarity.
☐	D056 Text classification with embeddings	Feed embeddings into a classifier instead of TF-IDF.
☐	D057 NER & POS tagging	Extract entities and tags with spaCy.
☐	D058 Sentiment analysis project	Full pipeline: clean → embed → classify → evaluate.
☐	D059 Intro to Hugging Face transformers	Load a pretrained model with <code>pipeline()</code> .
☐	D060 Mini project + review	Build a sentence-similarity search app.

SPRINT 07 · DAYS 061–070 · ATTENTION + TRANSFORMERS

☐	D061	Why attention — RNN limitations	List the specific failure modes attention solves.
☐	D062	Self-attention, step by step	Walk through the mechanism on a toy sentence.
☐	D063	Query, Key, Value math	Compute QKV attention scores by hand for a mini example.
☐	D064	Multi-head attention	Understand why multiple heads outperform one.
☐	D065	Positional encoding	Sinusoidal encodings — why order needs to be injected.
☐	D066	Transformer encoder architecture	Layer norm, residuals, feed-forward blocks.
☐	D067	Transformer decoder & encoder-decoder	Masked attention and cross-attention.
☐	D068	Implement scaled dot-product attention	Code it from scratch in PyTorch, no libraries.
☐	D069	Build a mini-transformer block	Stack your attention + FFN into a full block.
☐	D070	Mini project + review	Visualize attention weights on real sentences.

SPRINT 08 · DAYS 071–080 · GPT, BERT & TOKENIZERS

☐	D071	BERT architecture	Masked language modeling & next-sentence prediction.
☐	D072	Fine-tune BERT for classification	Fine-tune on a text classification dataset via HF.
☐	D073	GPT architecture	Autoregressive decoding and causal masking.
☐	D074	Byte-Pair Encoding internals	Understand subword tokenization step by step.
☐	D075	Build a tokenizer from scratch	Implement a basic BPE tokenizer in Python.
☐	D076	Sampling strategies	Greedy, beam search, top-k, top-p, temperature — compare outputs.
☐	D077	BERT vs GPT vs T5	Map encoder-only / decoder-only / encoder-decoder to use cases.
☐	D078	HF datasets & Trainer API	Fine-tune using the high-level Trainer class.
☐	D079	Prompting an open-source LLM	Run a local or hosted open model, write structured prompts.
☐	D080	Mini project + review	Fine-tune a small model on your own custom text data.

SPRINT 09 · DAYS 081–090 · LLM INTERNALS

☐	D081	Inference pipeline deep dive	Trace a prompt from tokens to logits to output text.
☐	D082	KV cache	Why it exists and how it speeds up autoregressive generation.
☐	D083	Context windows & position schemes	RoPE and ALiBi — intuition, not derivation.
☐	D084	Quantization basics	int8/int4, GGUF, bitsandbytes — trade-offs explained.
☐	D085	Running local LLMs	Set up Ollama or llama.cpp and run a 7B model locally.
☐	D086	LoRA & PEFT concepts	Why parameter-efficient fine-tuning works.
☐	D087	Serving basics	How a model gets exposed behind an API endpoint.
☐	D088	Latency, throughput & batching	Understand what production LLM serving optimizes for.
☐	D089	Token economics	Estimate real cost/latency trade-offs across model providers.
☐	D090	Mini project + review	Benchmark a local LLM against a hosted API model.

SPRINT 10 · DAYS 091–100 · EMBEDDINGS + FAISS

☐	D091	Embedding models deep-dive	Compare OpenAI, Cohere, and open-source embedding models.
☐	D092	Chunking strategies	Fixed-size vs semantic chunking for long documents.
☐	D093	FAISS basics	Flat, IVF, and HNSW index types explained.
☐	D094	Build a semantic search engine	Index and query documents with FAISS.
☐	D095	Evaluating retrieval quality	Recall@k and precision — measure your search engine.
☐	D096	Hybrid search	Combine keyword (BM25) with vector search.
☐	D097	Re-ranking with cross-encoders	Add a re-ranking stage to improve top results.
☐	D098	Metadata filtering	Filter vector search results by structured metadata.
☐	D099	Scaling ANN search	Trade-offs between exact and approximate nearest neighbors.
☐	D100	Mini project + review	Semantic search over your own document collection.

SPRINT 11 · DAYS 101–110 · VECTOR DATABASES

☐	D101	Vector DB landscape	Qdrant, Chroma, Pinecone, Weaviate — how they differ.
☐	D102	Set up Chroma locally	CRUD operations against a local Chroma collection.
☐	D103	Set up Qdrant	Collections, payloads, and filtering with Qdrant.
☐	D104	Indexing strategies in production	Choose index types for your data scale.
☐	D105	Multi-tenancy & namespaces	Isolate data per user/tenant in a vector DB.
☐	D106	Backup, persistence & scaling	Plan for durability and horizontal scale.
☐	D107	Connect DB to embedding pipeline	Wire ingestion straight into your vector store.
☐	D108	Benchmark Chroma vs Qdrant	Compare latency and recall on your own dataset.
☐	D109	Build an ingestion pipeline	Docs → chunks → embeddings → DB, fully automated.
☐	D110	Mini project + review	Ship a standalone document ingestion service.

SPRINT 12 · DAYS 111–120 · PRODUCTION RAG

☐	D111	RAG architecture end-to-end	Map the full retrieve-then-generate pipeline.
☐	D112	Prompt engineering for RAG	Design context-injection prompts that reduce hallucination.
☐	D113	Long & multi-document retrieval	Handle documents that exceed a single context window.
☐	D114	Citations & source attribution	Return answers with traceable source references.
☐	D115	Evaluating RAG quality	Faithfulness, relevance, and hallucination checks.
☐	D116	Conversation memory	Support multi-turn RAG conversations.
☐	D117	Guardrails & 'I don't know'	Design refusal behavior when retrieval is weak.
☐	D118	Streaming responses	Stream tokens to a frontend as they're generated.
☐	D119	Building the PDF chatbot backend	Wire ingestion, retrieval and generation together.
☐	D120	Capstone: PDF chatbot	Ship a working RAG chatbot end to end.

SPRINT 13 · DAYS 121–130 · FASTAPI + AI APIS

☐	D121	FastAPI basics	Routes, path params, query params.
☐	D122	Pydantic models & validation	Type-safe request/response schemas.
☐	D123	Async endpoints & background tasks	Non-blocking I/O for LLM calls.
☐	D124	Streaming responses (SSE)	Stream LLM tokens over server-sent events.
☐	D125	Authentication basics	API keys and JWT-based auth.
☐	D126	Connect FastAPI to your RAG pipeline	Expose your capstone RAG chatbot as an API.
☐	D127	Error handling, logging, rate limiting	Make the API production-safe.
☐	D128	OpenAPI docs & testing	Auto-generated docs plus automated tests.
☐	D129	Dockerizing a FastAPI app	Write a Dockerfile and run it in a container.
☐	D130	Mini project + review	Production-style REST API in front of your RAG app.

SPRINT 14 · DAYS 131–140 · AGENTS + LANGGRAPH

☐	D131	What is an agent — ReAct	Reasoning + acting loop explained with an example.
☐	D132	Tool / function calling	Give an LLM callable tools and parse its tool calls.
☐	D133	Intro to LangChain	Chains, tools, and basic agent constructs.
☐	D134	Intro to LangGraph	Model an agent as a state graph.
☐	D135	Building a multi-step agent	Planner + executor pattern on a real task.
☐	D136	Multi-agent orchestration	Coordinate multiple specialized agents.
☐	D137	Memory in agents	Short-term vs long-term memory strategies.
☐	D138	Agent evaluation & tracing	Debug agent runs with tracing tools.
☐	D139	Handling failures & retries	Make agent execution resilient.
☐	D140	Mini project + review	Ship a multi-agent research/task assistant.

SPRINT 15 · DAYS 141–150 · FINE-TUNING, DOCKER & MLOPS

☐	D141	Fine-tuning vs RAG vs prompting	Decide which technique fits which problem.
☐	D142	Full fine-tuning vs LoRA/QLoRA	Compare cost, quality and setup complexity.
☐	D143	Preparing a fine-tuning dataset	Clean and format data for supervised fine-tuning.
☐	D144	Fine-tune with LoRA	Fine-tune an open-source LLM on your dataset.
☐	D145	Docker fundamentals for ML apps	Images, containers, volumes — the essentials.
☐	D146	Docker Compose multi-container app	Wire together API + DB + vector store.
☐	D147	CI/CD basics for ML projects	Automate tests and deployments on push.
☐	D148	Monitoring & observability	Logging, metrics, and tracing in production.
☐	D149	Cloud deployment walkthrough	Deploy your app to a cloud provider.
☐	D150	Capstone: Deploy & document	Ship your full AI Engineer project with a portfolio writeup.

"Talk is cheap. Show me the code." – Linus Torvalds

150 days. 15 sprints. 1 portfolio-ready AI engineer.

Print this roadmap or keep it on a second screen. Tick a box every day —
momentum compounds faster than any single tutorial.